# Anomaly Detection & Time Series | Assignment

***Assignment Code: DA-AG-018***

## *Question 1: What is Anomaly Detection? Explain its types (point, contextual, and collective anomalies) with examples.*

**Anomaly detection** (outlier detection) is the process of identifying observations, events or patterns that deviate significantly from the expected or normal behaviour of a dataset. Anomalies are rare, but they often point to important events such as fraud, equipment failure, network intrusion, data-entry errors or medical emergencies. Methods can be statistical (Z-score, IQR), distance/density based (LOF, DBSCAN), tree based (Isolation Forest) or model based (ARIMA residuals, autoencoders).

**Types of anomalies:**

**1. Point anomaly:** A single data point that is abnormal compared to the rest of the data. *Example:* A credit card transaction of Rs. 5,00,000 when the customer's usual transactions are Rs. 500 to Rs. 5,000.

**2. Contextual (conditional) anomaly:** A data point that is abnormal only in a specific context (time, location, season), though it may be normal elsewhere. *Example:* A temperature of 30 degrees C is normal in summer but anomalous in January in Shimla; high electricity use at 3 AM in an office building.

**3. Collective anomaly:** A group of related data points that is abnormal together, even if each individual point looks normal. *Example:* A flat-line in an ECG for several seconds, or a sequence of many small bank withdrawals in a few minutes (a coordinated attack); a sudden sustained burst of network packets in a DDoS attack.

## *Question 2: Compare Isolation Forest, DBSCAN, and Local Outlier Factor in terms of their approach and suitable use cases.*

| Aspect | Isolation Forest | DBSCAN | Local Outlier Factor (LOF) |
|---|---|---|---|
| Approach | Builds random trees that split features randomly. Anomalies are few and different, so they are isolated in fewer splits (short path length). | Density-based clustering. Points with at least *min_samples* neighbours within radius *eps* form clusters; points in no cluster are noise (outliers). | Compares the local density of a point with the densities of its k nearest neighbours. LOF score much greater than 1 means the point is in a sparser area than its neighbours. |
| Type | Ensemble, tree based, global | Clustering, density based | Density based, local |
| Key parameters | n_estimators, contamination, max_samples | eps, min_samples | n_neighbors, contamination |
| Strengths | Fast (linear time), scales to large and high-dimensional data, no distance computation, no scaling needed | Finds arbitrary-shaped clusters, no need to specify number of clusters, labels noise automatically | Detects local outliers in data with clusters of varying density |
| Weaknesses | Weak for local anomalies in dense regions; random results unless seed fixed | Sensitive to eps; poor with varying densities and high dimensions | Slow on big data ($O(n^2)$); sensitive to k; needs scaling |
| Use cases | Fraud detection, server log monitoring, large tabular data, sensor data | Geospatial data (GPS points), customer segmentation with noise, finding isolated points | Fine-grained local anomalies, e.g. a slightly unusual transaction inside a dense normal cluster, network intrusion |

## *Question 3: What are the key components of a Time Series? Explain each with one example.*

A time series is a sequence of observations recorded over time. It is made of four components:

**1. Trend (T):** The long-term upward or downward movement of the series. *Example:* Airline passengers rising steadily from 1949 to 1960.

**2. Seasonality (S):** Regular, repeating patterns at a fixed period (daily, weekly, monthly, yearly). *Example:* Ice-cream sales peak every summer; airline traffic peaks every July/August.

**3. Cyclic component (C):** Rises and falls over irregular, longer periods (usually more than a year) linked to economic or business cycles, with no fixed length. *Example:* Real-estate prices following boom and recession cycles.

**4. Irregular / Residual / Noise (R):** Random, unpredictable fluctuations left after removing the other components. *Example:* A sudden dip in sales due to a strike or a natural disaster.

Models: **Additive:** Y = T + S + C + R (constant seasonal amplitude). **Multiplicative:** Y = T x S x C x R (seasonal amplitude grows with the level).

### *Question 4: Define Stationary in time series. How can you test and transform a non-stationary series into a stationary one?*

**Stationarity:** A time series is (weakly) stationary if its statistical properties do not change over time: the **mean is constant**, the **variance is constant**, and the **autocovariance depends only on the lag**, not on time. Such a series has no trend and no seasonality. Many models (AR, MA, ARIMA) assume stationarity.

**How to test:**

1. **Visual inspection:** plot the series, rolling mean and rolling standard deviation; a drifting mean or changing variance means non-stationary. ACF plot decaying very slowly is also a sign.

2. **Augmented Dickey-Fuller (ADF) test:** $H_0$ = series has a unit root (non-stationary). If p-value < 0.05, reject $H_0$ and treat the series as stationary.

3. **KPSS test:** $H_0$ = series is stationary. If p-value < 0.05, the series is non-stationary. Using both ADF and KPSS gives a more reliable conclusion.

**How to transform:**

1. **Differencing:** $y'_t = y_t - y_{t-1}$ removes trend; seasonal differencing $y_t - y_{t-s}$ (e.g. s = 12) removes seasonality. Repeat until the ADF test passes (this is the 'd' in ARIMA).

2. **Log or Box-Cox transformation:** stabilises growing variance.

3. **Detrending:** subtract a fitted trend line or moving average.

4. **Decomposition:** remove trend and seasonal components and model the residual.

### *Question 5: Differentiate between AR, MA, ARIMA, SARIMA, and SARIMAX models in terms of structure and application.*

| Model | Structure | Application |
|---|---|---|
| AR(p) | Current value regressed on its own p past values: $y_t = c + f_1 y_{t-1} + ... + f_p y_{t-p} + e_t$. Requires a stationary series. | Series with strong dependence on its own past, e.g. temperature, stock returns with momentum. |
| MA(q) | Current value is a linear combination of past q forecast errors: $y_t = c + e_t + q_1 e_{t-1} + ... + q_q e_{t-q}$. Stationary series. | Series driven by short-lived random shocks, e.g. sudden demand spikes that fade. |
| ARIMA(p,d,q) | AR + MA + differencing (I) of order d to handle non-stationarity (trend). | Non-seasonal data with trend, e.g. yearly GDP, stock price levels. |
| SARIMA(p,d,q)(P, D,Q)s | ARIMA plus seasonal AR, differencing and MA terms with seasonal period s. | Data with trend and seasonality, e.g. monthly airline passengers (s = 12), monthly sales. |
| SARIMAX | SARIMA plus **exogenous** external regressors (X). | When outside factors matter, e.g. energy demand with temperature, sales with promotions or holidays. |

### *Question 6: Load a time series dataset (AirPassengers), plot the original series, and decompose it into trend, seasonality, and residual components.*

**Code:**

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
df = pd.read_csv(url, parse_dates=["Month"], index_col="Month")
df.columns = ["Passengers"]
print(df.head())
print("Shape:", df.shape)

# 1. Plot the original series
df["Passengers"].plot(figsize=(10, 4), title="Monthly Airline Passengers (1949-1960)")
plt.ylabel("Passengers")
plt.show()

# 2. Decompose (multiplicative, because seasonal swings grow with the level)
result = seasonal_decompose(df["Passengers"], model="multiplicative", period=12)
fig = result.plot()
fig.set_size_inches(10, 8)
plt.tight_layout()
plt.show()
```

**Output:** [Paste screenshot of the output from your notebook here. You will see (1) the line plot of the series rising from about 100 to 600 passengers with growing yearly peaks, and (2) four panels: Observed, Trend (steady upward), Seasonal (repeating 12-month pattern with peak in July/August) and Residual (small random noise around 1).]

**Interpretation:** AirPassengers has a clear upward trend and strong yearly seasonality whose amplitude grows with the level, so a multiplicative decomposition is appropriate.

### *Question 7: Apply Isolation Forest on a numerical dataset (NYC Taxi Fare) to detect anomalies. Visualize the anomalies on a 2D scatter plot.*

**Code:**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

# NYC Taxi Fare data (Kaggle: new-york-city-taxi-fare-prediction, train.csv)
df = pd.read_csv("train.csv", nrows=100000).dropna()

# Keep valid NYC coordinates and sensible fares
df = df[(df.pickup_latitude.between(40, 42)) & (df.dropoff_latitude.between(40, 42)) &
        (df.pickup_longitude.between(-75, -72)) & (df.dropoff_longitude.between(-75, -72)) &
        (df.fare_amount > 0)]

# Trip distance in km (haversine formula)
def haversine(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    a = np.sin((lat2 - lat1) / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin((lon2 - lon1) / 2) ** 2
    return 6371 * 2 * np.arcsin(np.sqrt(a))

df["distance_km"] = haversine(df.pickup_latitude, df.pickup_longitude,
                              df.dropoff_latitude, df.dropoff_longitude)

X = df[["distance_km", "fare_amount"]]
X_scaled = StandardScaler().fit_transform(X)

iso = IsolationForest(n_estimators=200, contamination=0.01, random_state=42)
df["anomaly"] = iso.fit_predict(X_scaled)          # -1 = anomaly, 1 = normal
print(df["anomaly"].value_counts())

plt.figure(figsize=(9, 6))
normal, anom = df[df.anomaly == 1], df[df.anomaly == -1]
plt.scatter(normal.distance_km, normal.fare_amount, s=4, c="steelblue", label="Normal")
plt.scatter(anom.distance_km, anom.fare_amount, s=14, c="red", label="Anomaly")
plt.xlabel("Trip distance (km)"); plt.ylabel("Fare amount ($)")
plt.title("Isolation Forest - NYC Taxi Fare Anomalies")
plt.legend(); plt.show()
```

**Output:** [Paste screenshot from your notebook: the value counts (about 1% flagged as -1) and the scatter plot of distance vs fare with red anomaly points.]

**Interpretation:** Isolation Forest flags trips whose fare is unusual for their distance, e.g. very high fares on short trips or extremely long, expensive trips. These can be data errors or special trips (airport flat fares, tolls).

### *Question 8: Train a SARIMA model on the monthly airline passengers dataset. Forecast the next 12 months and visualize the results.*

**Code:**

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
df = pd.read_csv(url, parse_dates=["Month"], index_col="Month")
y = df["Passengers"].asfreq("MS")

# SARIMA(1,1,1)(1,1,1,12): d=1 and D=1 remove trend and yearly seasonality
model = SARIMAX(y, order=(1, 1, 1), seasonal_order=(1, 1, 1, 12))
fit = model.fit(disp=False)
print(fit.summary())

# Forecast next 12 months with 95% confidence interval
fc = fit.get_forecast(steps=12)
mean, ci = fc.predicted_mean, fc.conf_int()

plt.figure(figsize=(10, 5))
plt.plot(y, label="Observed")
plt.plot(mean, color="red", label="Forecast (next 12 months)")
plt.fill_between(ci.index, ci.iloc[:, 0], ci.iloc[:, 1], color="pink", alpha=0.4, label="95% CI")
plt.title("SARIMA Forecast - Airline Passengers")
plt.xlabel("Date"); plt.ylabel("Passengers"); plt.legend(); plt.show()
print(mean.round(0))
```

**Output:** [Paste screenshot: model summary, and the plot with the observed series, red 12-month forecast and shaded 95% confidence interval.]

**Interpretation:** SARIMA(1,1,1)(1,1,1,12) captures the trend and the 12-month seasonality; the forecast continues the upward trend with the same seasonal peaks in summer, and the confidence band widens further into the future.

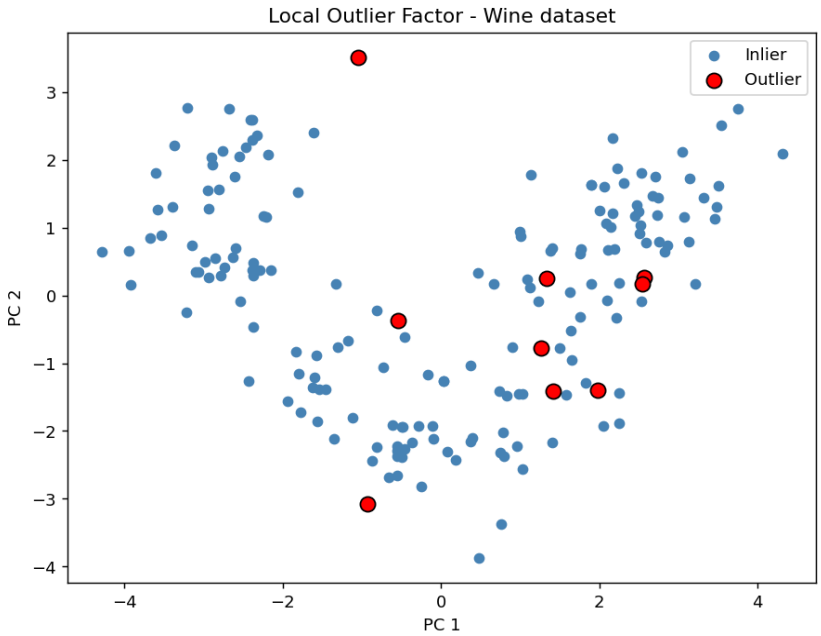
## *Question 9: Apply Local Outlier Factor (LOF) on any numerical dataset to detect anomalies and visualize them using matplotlib.*

**Code:**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import LocalOutlierFactor

# Numerical dataset: Wine (178 samples, 13 numeric features)
X = load_wine().data
X_scaled = StandardScaler().fit_transform(X)

lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05)
labels = lof.fit_predict(X_scaled)                # -1 = outlier, 1 = inlier
scores = -lof.negative_outlier_factor_            # higher = more anomalous
print("Total points:", len(X), "| Outliers found:", (labels == -1).sum())
print("Top 5 LOF scores:", np.sort(scores)[-5:].round(2))

# 2D view using PCA
X2 = PCA(n_components=2).fit_transform(X_scaled)
plt.figure(figsize=(8, 6))
plt.scatter(X2[labels == 1, 0], X2[labels == 1, 1], c="steelblue", s=30, label="Inlier")
plt.scatter(X2[labels == -1, 0], X2[labels == -1, 1], c="red", s=80,
            edgecolors="k", label="Outlier")
plt.xlabel("PC 1"); plt.ylabel("PC 2")
plt.title("Local Outlier Factor - Wine dataset")
plt.legend(); plt.show()
```

**Output:**

```
Total points: 178 | Outliers found: 9
Top 5 LOF scores: [1.55 1.65 1.72 1.74 1.78]
```




**Interpretation:** LOF flagged 9 of 178 wine samples (about 5%, as set by contamination) as outliers. They lie at the edges of the PCA scatter where local density is lower than that of their neighbours (LOF scores up to about 1.8).

## *Question 10: Real-time data science workflow for a power grid monitoring company (15-minute data: timestamp, region, weather, energy usage).*

**1. Data pipeline:** Smart-meter data streams into Kafka (or a cloud queue), is cleaned (missing values, duplicates, unit checks) and stored in a time-series database. Features: lag values (previous 15 min, same time yesterday and last week), rolling mean/std, hour, day of week, holiday flag, temperature, humidity, region.

**2. Anomaly detection (Isolation Forest / LOF / DBSCAN):** I would use **Isolation Forest** as the main detector, trained per region on normal history and applied to each new 15-minute window of features (usage, change from previous interval, deviation from the forecast residual). Reasons: it is fast, scales to many regions, works on streaming micro-batches and needs no distance matrix. LOF is slower and needs the whole reference set, so it is suited to offline investigation of local anomalies. DBSCAN is not ideal for live scoring because it must recluster; it is useful for offline grouping of anomaly events. Because usage has strong daily seasonality, detection is done on **forecast residuals**, which separates true spikes/drops from normal peak hours. Alerts are raised when the anomaly score crosses a threshold, with a persistence rule (for example 2 consecutive windows) to cut false alarms.

**3. Short-term forecasting model:** I would choose **SARIMAX**. Energy demand has daily seasonality (96 intervals per day) and depends strongly on external weather such as temperature, which only SARIMAX can include as exogenous variables (ARIMA has no seasonality; SARIMA has no external inputs). Steps: check stationarity with ADF, difference if needed, choose orders from ACF/PACF or auto-search by AIC, and use forecast temperature as the exogenous input for the next 1 to 24 hours.

**4. Validation and monitoring:** Use **walk-forward (rolling-origin) cross-validation**, never random splits. Metrics: MAE, RMSE and MAPE against a naive baseline (same time yesterday). In production, track forecast error daily, residual ACF, anomaly rate and score drift; detect **data/concept drift** (new seasons, new consumers) and retrain on a schedule (e.g. weekly) or when error exceeds a threshold. Review flagged anomalies with operators and feed confirmed labels back to tune the contamination threshold. Keep dashboards and alerts on model health.

**5. Business value:** Accurate forecasts support generation scheduling, power purchase planning and load balancing, reducing penalties and cost of buying at peak prices. Early anomaly alerts help detect equipment faults, outages, theft or meter tampering and cyber-attacks, so crews respond sooner. Overall: higher reliability, lower operating cost, better demand-response and capacity planning.

**Code:**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import IsolationForest
from statsmodels.tsa.statespace.sarimax import SARIMAX

# ---- 1. Simulated 15-minute grid data (replace with real feed) ----
rng = np.random.default_rng(42)
idx = pd.date_range("2025-01-01", periods=96 * 14, freq="15min")        # 14 days
hour = idx.hour + idx.minute / 60
temp = 25 + 8 * np.sin((hour - 9) / 24 * 2 * np.pi) + rng.normal(0, 0.5, len(idx))
usage = 300 + 80 * np.sin((hour - 6) / 24 * 2 * np.pi) + 4 * temp + rng.normal(0, 8, len(idx))
usage[[200, 600, 900]] += [250, -200, 300]                              # injected spikes/drops
data = pd.DataFrame({"usage": usage, "temp": temp}, index=idx)

# ---- 2. Short-term forecasting: SARIMAX with weather as exogenous input ----
train, test = data.iloc[:-96], data.iloc[-96:]                          # forecast last day
model = SARIMAX(train["usage"], exog=train[["temp"]],
                order=(2, 0, 1), seasonal_order=(1, 0, 0, 96))
fit = model.fit(disp=False, maxiter=50)
pred = fit.get_forecast(steps=96, exog=test[["temp"]]).predicted_mean
mape = (abs(test["usage"] - pred) / test["usage"]).mean() * 100
print(f"Next-24h forecast MAPE: {mape:.2f}%")

# ---- 3. Anomaly detection on model residuals + rolling window ----
feat = pd.DataFrame({"resid": fit.resid,
                     "roll_mean": data["usage"].rolling(4).mean().loc[train.index],
                     "diff": data["usage"].diff().loc[train.index]}).dropna()
iso = IsolationForest(contamination=0.005, random_state=42).fit(feat)
feat["anomaly"] = iso.predict(feat)
print("Anomalies flagged:", (feat.anomaly == -1).sum())

# ---- 4. Visualise ----
fig, ax = plt.subplots(2, 1, figsize=(11, 7))
ax[0].plot(test["usage"], label="Actual"); ax[0].plot(pred, "r--", label="Forecast")
ax[0].set_title("Next-day demand forecast (SARIMAX)"); ax[0].legend()
ax[1].plot(data["usage"], lw=0.7)
a = feat[feat.anomaly == -1].index
ax[1].scatter(a, data.loc[a, "usage"], c="red", label="Anomaly")
ax[1].set_title("Detected spikes/drops (Isolation Forest)"); ax[1].legend()
```

```
plt.tight_layout(); plt.show()
```

**Output:** [Paste screenshot: forecast MAPE, number of anomalies flagged, and the two plots (actual vs forecast, and usage with red anomaly points).]